

Chapter 3

Nearest neighbor-based classifiers

Assoc. Prof. Dr. Duong Tuan Anh
Faculty of Computer Science and Engineering,
HCMC Univ. of Technology
3/2015

Outline

- ❑ Introduction
- ❑ Nearest Neighbor algorithm
- ❑ Variants of the NN algorithm
- ❑ Data Reduction
- ❑ Prototype selection

1. Introduction

- One of the simplest decision procedures that can be used for classification: the ***nearest neighbor*** algorithm.
- The nearest neighbor based classifiers use some or all patterns in the training set to classify a test pattern.
- These classifiers involve finding the ***similarity*** between the test pattern and every pattern in the training set.
- ***Lazy learners***: do less work when a training pattern is presented and more work when making a classification.
- NN classifier $\in$ lazy learner, ***instance-based learner***

2. Nearest Neighbor algorithm

- The nearest neighbor (NN) algorithm assign to a test pattern the class label of its closest neighbor.
- Let there be n training patterns, $(X_1, c_1), (X_2, c_2), \dots, (X_n, c_n)$, where X_i is of dimension d and c_i is the class label of i th pattern.
- If P is the test pattern, then if
$$d(P, X_k) = \min \{d(P, X_i)\} \quad i = 1, 2, \dots, n$$
pattern P is assigned to the class k associated with X_k .
- We have to use some **distance measure**, e.g. **Euclidean distance** to measure the “closeness” between the test pattern P to some pattern in the training set.
- The Euclidean distance between two tuples, say, $X_1 = (x_{11}, x_{12}, \dots, x_{1n})$ and $X_2 = (x_{21}, x_{22}, \dots, x_{2n})$ is

$$\text{dist}(X_1, X_2) = \sqrt{\sum_{i=1}^n (x_{1i} - x_{2i})^2}$$

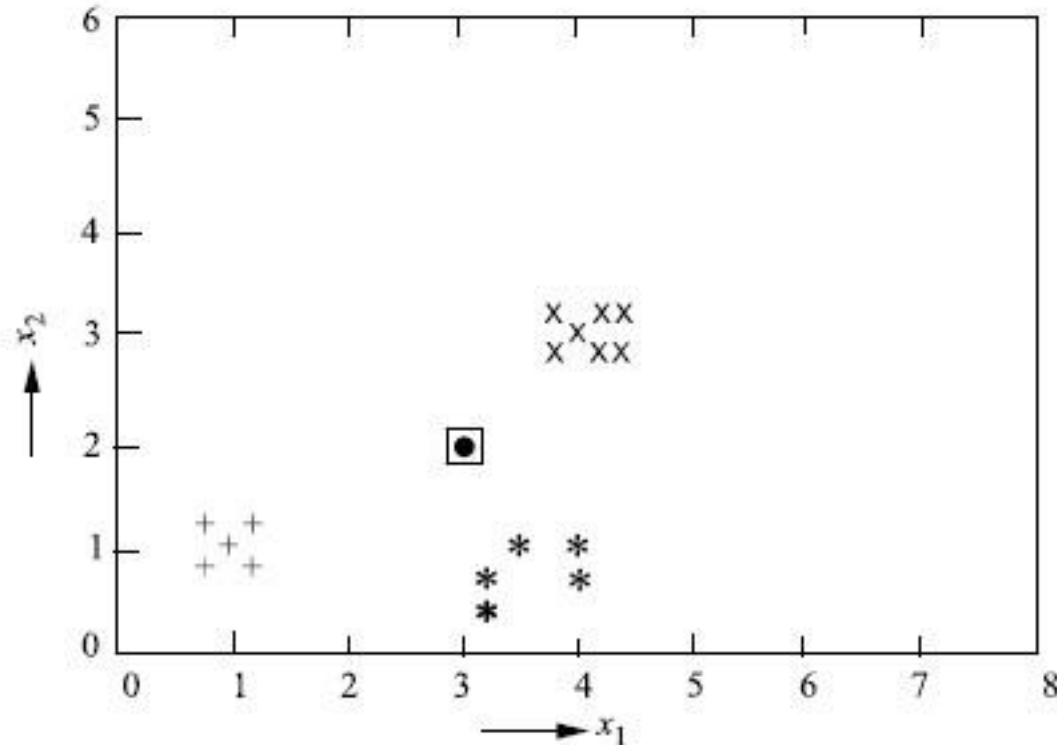
Example

Figure 3.1
Example dataset

+: 1

X: 2

*: 3



The training set: $X_1 = (0.8, 0.8, 1)$, $X_2 = (1.0, 1.0, 1)$, $X_3 = (1.2, 0.8, 1)$,
 $X_4 = (0.8, 1.2, 1)$, $X_5 = (1.2, 1.2, 1)$, $X_6 = (4.0, 3.0, 2)$, $X_7 = (3.8, 2.8, 2)$,
 $X_8 = (4.2, 2.8, 2)$, $X_9 = (3.8, 3.2, 2)$, $X_{10} = (4.2, 3.2, 2)$, $X_{11} = (4.4, 2.8, 2)$,
 $X_{12} = (4.4, 3.2, 2)$, $X_{13} = (3.2, 0.4, 3)$, $X_{14} = (3.2, 0.7, 3)$, $X_{15} = (3.8, 0.5, 3)$,
 $X_{16} = (3.5, 1.0, 3)$, $X_{17} = (4.0, 1.0, 3)$, $X_{18} = (4.0, 0.7, 3)$

- If the test pattern P is (3.0, 2.0), we have to find the distance from P to all the training patterns.
- Here we use Euclidean distance.
$$\text{dist}(X_1, P) = \text{sqrt}((0.8-3.0)^2 + (0.8-2.0)^2) = 2.51$$
- After calculating the distance from all the training patterns to P , the closest neighbor of P is X_{16} , $\text{dist}(X_{16}, P) = 1.12$.
- Since X_{16} belongs to Class 3, P is classified as belonging to Class 3.
- **Note:** We should **normalize** the values of each attribute before calculating Euclidean distance. This helps prevent attributes with initially large ranges from outweighing attribute with initially smaller ranges. **Min-Max normalization** can be used to transform a value v of a numeric attribute A in the range $[\min_A, \max_A]$ to v' in the range $[\text{new_min}_A, \text{new_max}_A]$ by computing:

$$v' = \frac{v - \min_A}{\max_A - \min_A} (\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A$$

Z-score normalization

- In *z-score normalization* (or *zero-mean normalization*), the values for an attribute A are normalized based on the *mean* and *standard deviation* of A .
- A value, v , of A is normalized to v' by computing:

$$v' = (v - \bar{A}) / \sigma_A$$

where $\bar{A}$ and σ_A are the *mean* and *standard deviation*, respectively of attribute A .

If $v = \bar{A}$ then $v' = 0$

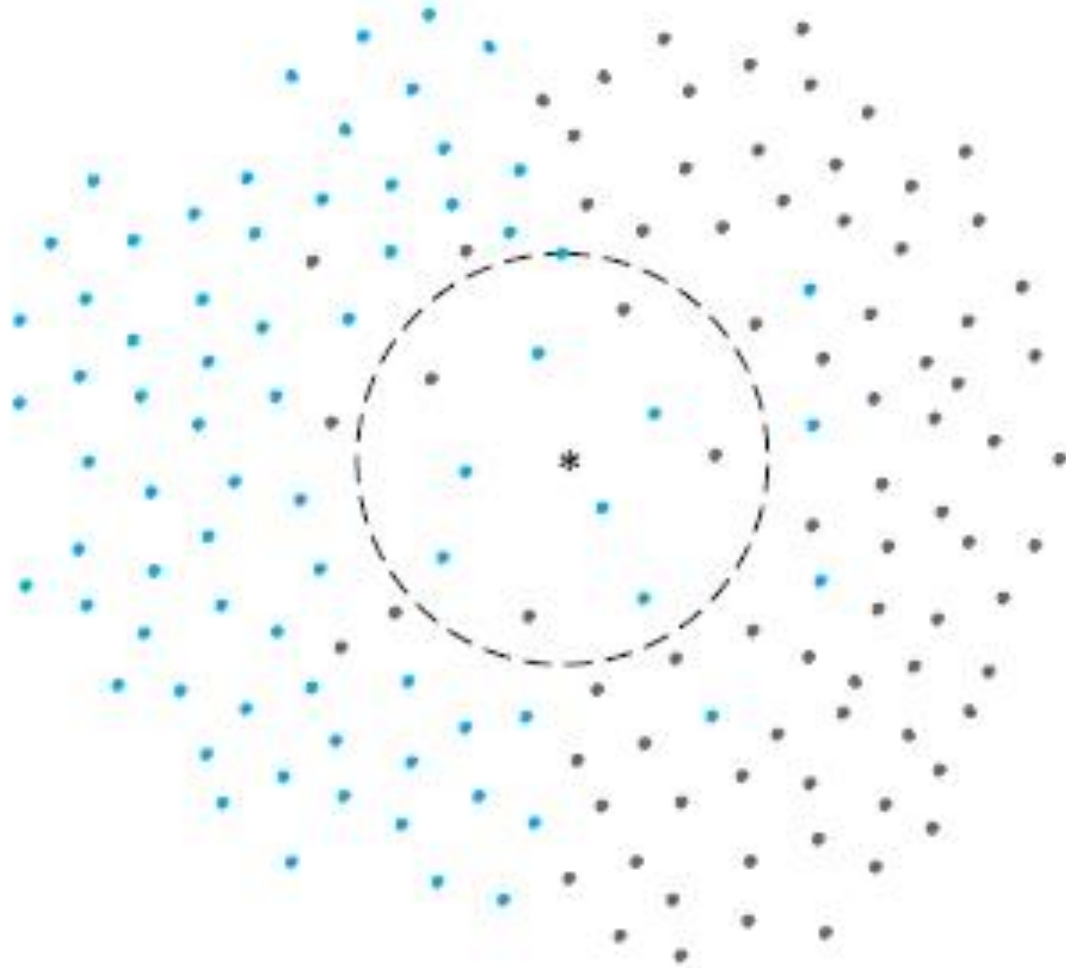
If $v = \bar{A} + \sigma_A$ then $v' = 1$

3. Variants of the NN algorithm

K-Nearest Neighbor (k-NN) algorithm

- ❑ Instead of finding just one nearest neighbor, **k neighbors are found**.
- ❑ The **majority class** of these k nearest neighbors is the class label assigned to the new pattern.
- ❑ This method reduces the error in classification when the training patterns are noisy.
- ❑ The value of k is crucial. With the right value of k , k -NN is better than 1-NN.
- ❑ The value of k can be determined by experiment. For different values of k :
 - A number of patterns taken out from the training set (as a validation set) can be classified using the remaining training pattern as the training set. Measure the classification error.
 - k can be chosen as the value which gives the least error in classification.

An illustration of k-NN



Example

- In the example in Figure 3.1, if k is 5, the five neighbors of P are X_{16} , X_7 , X_{14} , X_6 and X_{17} . The majority class of these five patterns is class 3.
- In Figure 3.1, if P is the pattern (4.2, 1.8), its nearest neighbor is X_{17} and it would be classified as in Class 3 if 1-NN is used. If the 5 nearest neighbors are taken, they are X_{17} and X_{16} (in Class 3) and X_8 , X_7 , and X_{11} (in Class 2). Following the majority class rule, the pattern P will be in Class 2.

Note: K-NN is among top 10 algorithms in Data Mining

Modified k -Nearest Neighbor algorithm

- This algorithm is similar to k -NN. But **these k nearest neighbors are weighted according to their distance** from the test pattern.
- It is called distance-weighted k -nearest neighbor algorithm.
- Each of the neighbors is associated with the weight w :

$$w_j = \begin{cases} (d_k - d_j)/(d_k - d_1) & \text{if } d_k \neq d_1 \\ 1 & \text{if } d_k = d_1 \end{cases} \quad (3.1)$$

where $j = 1, \dots, k$ and d_j is the distance from the test pattern to its j -th nearest neighbor. The value of w_j varies from 0 to 1.

The Modified k -NN algorithm assigns the test pattern P to that class for which the **weights** of the representatives among the k nearest neighbors sums to the greatest value.

The Modified k -NN employs a *weighted majority rule*.

Example

- Consider $P = (3.0, 2.0)$ in Figure 3.1. For the five nearest points, the distances from P are

$$d(P, X_{16}) = 1.12; d(P, X_7) = 1.13; d(P, X_{14}) = 1.32; d(P, X_6) = 1.41; \\ d(P, X_{17}) = 1.41;$$

The values of w will be

$$w_{16} = (1.41 - 1.12)/(1.41 - 1.12) = 1$$

$$w_7 = (1.41 - 1.13)/(1.41 - 1.12) = 0.97$$

$$w_{14} = (1.41 - 1.32)/(1.41 - 1.12) = 0.31$$

$$w_6 = 0$$

$$w_{17} = 0$$

Summing up for each class, Class 1 sums to 0, Class 2 to which X_7 and X_6 belong sums to 0.97 and Class 3 to which X_{16} , X_{14} and X_{17} belong sums to **1.31**. Therefore, P belongs to Class 3.

4. Data reduction

- The efficiency of the classification is dependent on the training data. A larger training data may lead to better classification but require a longer computation time.
- For classifiers such as neural networks or decision tree, it is only the design time which increases. Once the design is complete, the classification of new patterns will be fast.
- For k -NN classifier, every new pattern has to be compared to all the training patterns. Hence, the time to classify each pattern may be large.
- So, ***reducing the number of training patterns*** will lead to faster classification. (***prototype selection***)
- Another option: ***feature selection*** where the dimensionality of the patterns is reduced to a subset of the features.

5. Prototype selection

- Given a set of samples, a **prototype** would be a sample which represents or is a typical sample for a large number of samples in the set.
- Ex: Given a set of samples, the *centroid* of the set could be a prototype of the set.
- We can have a number of prototypes which collectively represent a set of samples.
- **Prototype selection** finds the sample or set of samples to represent a larger set of samples.
- Prototype selection provides a solution to the problem of high computation cost and space requirement in k -NN algorithm.
- Prototype selection is the process of reducing the training set used for classification.

- Formally, let $\mathbf{X} = \{(X_1, c_1), (X_2, c_2), \dots, (X_n, c_n)\}$ be the given labelled training set. Prototype selection is the process of obtaining $\mathbf{X}' = \{X^1, X^2, \dots, X^k\}$ from $\mathbf{X}$ such that:
 - $k < n$
 - Either $\mathbf{X}'$ is a subset of $\mathbf{X}$ or each X^i , $1 \leq i \leq k$ is obtained from the patterns in $\mathbf{X}$.
- Prototype selection must not make the classification accuracy decrease.
- There are some methods of prototype selection:
 - minimal distance classifier,
 - condensed nearest neighbor algorithm
- Prototype selection methods can be classified according to the following criteria:
 - Algorithms which select prototypes already available in the training set, i.e., $\mathbf{X}' \subset \mathbf{X}$. (*Prototype selection*)
 - Algorithms which make use of the training set and produce prototypes which are different from those in the training set, i.e. $\mathbf{X}' \not\subset \mathbf{X}$. (*Prototype abstraction*)

Minimal distance classifier

- One simple prototype selection strategy is to use the minimum distance classifier.
- Each class is represented by the sample *mean* or *centroid* of all the samples in the class. This method select only one prototype to represent a class.
- If $X_{i1}, X_{i2}, \dots, X_{iN}$ are the N training patterns for the class i , then the representative sample will be.

$$C_i = \frac{\sum_{j=1}^N X_{ij}}{N}$$

In order to classify a test pattern P , if C_k is the centroid closest to P , it is assigned the class label k of which C_k is the representative pattern.

Example

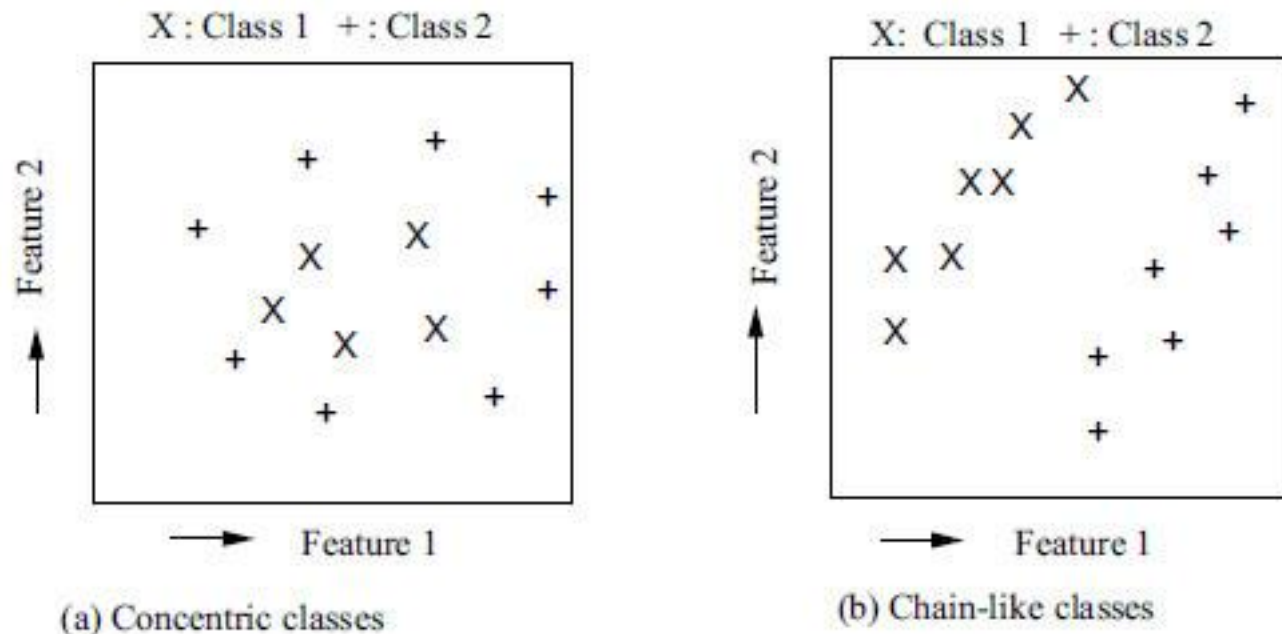
- Consider the example data set in Figure 3.1. The training set consists of 5 patterns of Class 1, 7 examples of Class 2 and 6 examples of Class 3.
- The centroid for class 1 can be computed from the patterns X_1, X_2, X_3, X_4, X_5 . The centroid will be (1.0, 1.0).
- Similarly, the centroid for Class 2 is (4.11, 3).
- The centroid for Class 3 is (3.62, 0.72).
- Consider a test pattern P at (3.0, 2.0). To determine its class, we find the distance of P from the centroids of the three classes.
From P to the centroid of Class 1, the distance is 2.24.
From P to the centroid of Class 2, the distance is 1.49
From P to the centroid of Class 3, the distance is 1.42.
Since P is closest to the centroid of Class 3, it is classified as belonging to Class 3 according to MDC.

Note: MDC belongs to *Prototype abstraction*

Condensation algorithms

- MDC is a time-efficient algorithm, it fails to give good results when the centroid is not a good representative of the patterns in a class.
- This happens when the classes are *chain-like* and elongated in one direction or when two or more classes are *concentric* (see Figure 3.2).
- In such cases, it may be better to use more than one representative pattern from each class.
- In this direction: there are some methods for prototype selection, e.g., *condensed nearest neighbor* (CNN)

Figure 3.2



Condensed nearest neighbor algorithm

Let *Train* be the set of n training pairs given by

$$\text{Train} = \{(X_1, c_1), (X_2, c_2), \dots, (X_n, c_n)\}$$

Let *Condensed* be the set which is empty initially.

- Step 1: Select the first sample from *Train* and add it to *Condensed*. Let *Reduced* be the set $\text{Train} - \text{Condensed}$.
- Step 2: Select the first sample from *Reduced* and find the nearest neighbor of the sample in *Condensed*. If the class label associated with the nearest neighbor and the selected sample are different, then add the sample to *Condensed*. Delete the selected pattern from *Reduced*.
- Step 3: Repeat step 2 until *Reduced* is empty.
- Step 4. Set $\text{Reduced} = \text{Train} - \text{Condensed}$. Repeat Steps 2 and 3.
- Step 5: Stop if there is no change in *Condensed* (or *Reduced*) during two successive iterations of Step 4 else iterate through Steps 2, 3 and 4.

Note: Condensed Nearest Neighbor belongs to Prototype Selection

Example

- Consider the training set of 11 samples belonging to 3 classes. Class 1 (X), Class 2 (O) and Class 3 (+).
- $X_1 = (1.0, 1.0, 1)$,
 $X_2 = (1.0, 2.0, 1)$, $X_3 = (1.5, 1.5, 1)$,
 $X_4 = (2.0, 2.0, 1)$, $X_5 = (3.0, 2.0, 2)$,
 $X_6 = (4.0, 2.0, 2)$, $X_7 = (4.0, 3.0, 2)$,
 $X_8 = (5.0, 2.5, 2)$, $X_9 = (2.0, 3.0, 3)$,
 $X_{10} = (3.0, 3.5, 3)$, $X_{11} = (2.0, 4.0, 3)$.

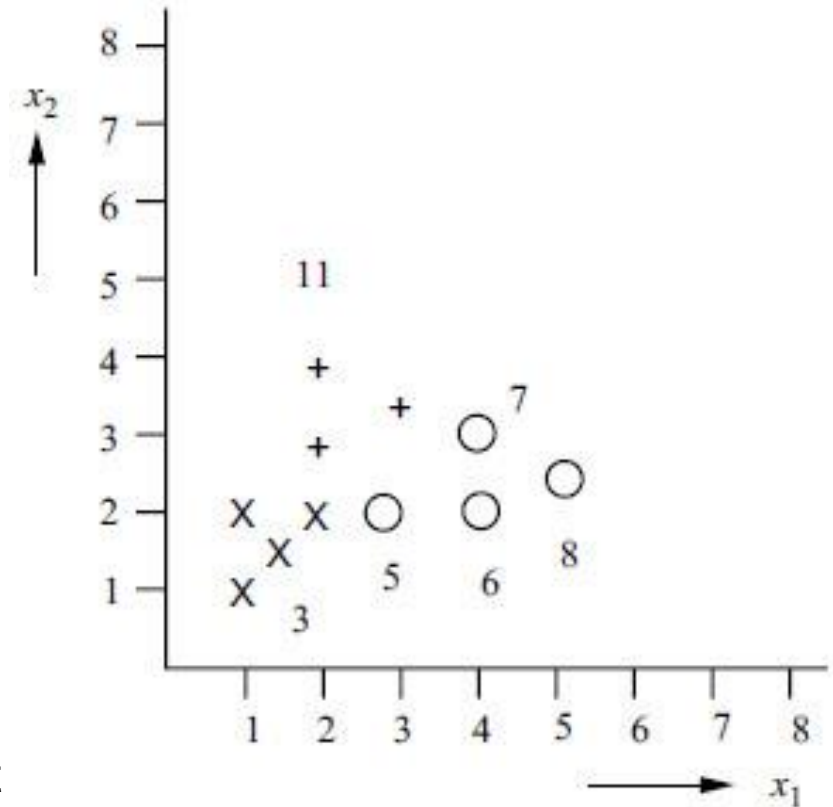


Figure 3.3 Example dataset

- The patterns are sent to the algorithm in order from X_1 to X_{11} .
- First, X_1 is put into the set *Condensed*. Since only X_1 is in *Condensed*, X_2 , X_3 and X_4 are closed to X_1 and since they have the same class as X_1 , nothing is done. Since X_5 has the class different from the class of X_1 , it is added to *Condensed*. Now X_6 is compared to X_1 and X_5 . Since it is closer to X_5 which has the same label as X_6 , nothing is done. X_7 and X_8 are closer to X_5 which has the same label, and hence they are not included in *Condensed*. Basing on the distances of X_9 to X_1 and X_9 to X_5 , we see that X_9 is closest to X_5 of all the patterns in *Condensed*. Since X_9 has a different label from X_5 , it is included in *Condensed*. Now *Condensed* has X_1 , X_5 and X_9 . The patterns X_{10} and X_{11} are closest to X_9 in *Condensed* and are therefore not included in *Condensed*. This completes one iteration.
- In the 2nd iteration, X_2 and X_3 are closest to X_1 which has the same label. They are not included in *Condensed*. X_4 is equidistant from X_5 and X_9 . If X_4 is taken to be closest to X_5 to break the tie, since its label is different from X_5 , it is included in *Condensed*. X_{10} and X_{11} are closest to X_9 which has the same label and are not included in *Condensed*.
- In the next iteration, since there is no change in *Condensed*, the *Condensed* set consists of X_1 , X_4 , X_5 and X_9 .

Pruning techniques

- **Pruning removes samples which are noisy** and reduces overlap among classes, giving an improved performance compared to the original data set.
- There are many existing pruning techniques.
- Here, we introduce ***Naïve Rank Numerosity Reduction*** method proposed by Xi et al., 2006.

Naïve Rank Reduction method

- The algorithm works in two steps: *ranking* and *thresholding*.
- We **first assigns ranks to all samples** based on their contribution in classification on the training set. Then we *use a threshold n to decide how many instances to keep* and simply keep these n highest ranked instances for future classification.
- In the *ranking* step, we begin by removing duplicated instances, if any. Then we apply one-nearest-neighbor classification on the training set.
- We give lowest ranks to misclassified instances since these instances tend to be noisy and may change the ranking of other instances.
- For correctly classified instances, we assign their ranks by the following formula:

Naïve rank reduction method (cont.)

$$\text{rank}(x) = \sum_j \begin{cases} 1 & \text{if } \text{class}(x) = \text{class}(x_j) \\ -2 & \text{otherwise} \end{cases}$$

where x_j is the instance having x as its nearest neighbor.

If two instances have the same rank, break the tie by assign different priorities to them. The priority of an instance x is calculated by:

$$\text{priority}(x) = \sum_j \frac{1}{d(x, x_j)^2}$$

where x_j is the instance having x as its nearest neighbor and $d(x, x_j)$ is the distance between instances x and x_j .

- We attempt to keep *good* instances by giving *positive* value to the instances that help correctly classify other data and giving more *negative* value to those that misclassify others.
- If two instances have the same rank, the one that is far from its nearest neighbor will have lower priority and will be discarded first.
- Note: The algorithm ranks the instance iteratively. Assuming the training set size is N , the algorithm will run N times, where in **each round it disregards the instance with lowest rank and continues to re-rank the rest** of the instances.
- Given the training set T , the algorithm will store the instances in list S according to their ranks.

Ranking step

Function $S = \text{Naïve_Rank_Reduction}(T)$

1. Remove any duplicate instances from T
2. Apply 1-NN classification on T ;
3. $N = \text{size of } T$
4. $S = \emptyset$;
5. $\text{loop_num} = 0$;
6. **while** $\text{loop_num} < N$ **do**
7. **for** each instance x_i in $T - S$
8. $\text{rank}(x_i)$ is calculated
9. **endfor**
10. Adjust the rank ties by priorities, if any
11. $\text{worst_index} = \arg_i (\min(\text{rank}[x_i] \& \min(\text{priority}[x_i])))$;
12. **push**($S, x_{\text{worst_index}}$) //Discard instance $x_{\text{worst_index}}$
13. $\text{loop_num} = \text{loop_num} + 1$
14. **endwhile**

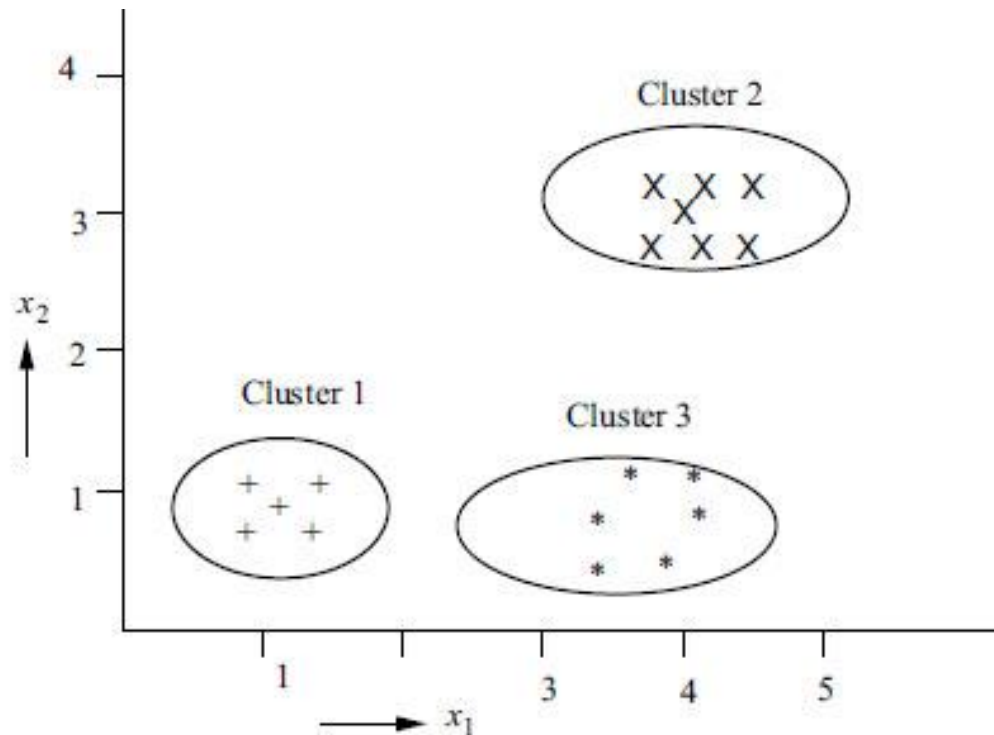
Thresholding step

- In the *thresholding* step, we keep the n highest ranking instances in S as the training set for future classification, when n is the threshold given by the user.
- This threshold may be given based on the maximum space or time the user has for a problem.
- Note: Naïve Rank Reduction belongs to *Prototype Selection*

Clustering methods in data reduction

- We can obtain representative patterns from a given data set by selecting clusters of high density data regions in multi-dimensional space and replacing each such region by its representative (e.g. the *centroid* of the data in the region).
- Clustering subdivides the patterns into groups so that patterns in the same group are similar while patterns belonging to different groups are dissimilar according to some criteria.
- It is also possible to have more clusters for each class.
- Example: The patterns in Figure 3.4 can be divided into three clusters, one for each class. If the centroid is the representative pattern of a cluster, Cluster 1 can be represented by the point $C_1 = (1.0, 1.0)$, Cluster 2 can be represented by the point $C_2 = (4.11, 3)$, and Cluster 3 can be represented by the point $C_3 = (3.62, 0.72)$,

Figure 3.4 Clustering of patterns given in Figure 3.1



Now if there is a test pattern P at $(4.2, 1.8)$, the distance to the three centroids is

$$d(C_1, P) = 3.30; \quad d(C_2, P) = 1.20; \quad d(C_3, P) = 1.23$$

Using the nearest neighbor rule, the pattern P will be classified as in Class 2.

Tools for k-NN Classifier

- K-nearest neighbor Classifier is available in:
 - ❑ MATLAB
 - ❑ The data mining software Weka
 - ❑ RapidMiner
 - ❑ Scikit-learn

Conclusions

- The nearest-neighbor classifiers are appealing and effective.
- The drawback is in the implementation: high computational time and space for large training data.
- To reduce the computation time, various approaches have been proposed which help retrieve the nearest neighbors in a short time (e.g. using the support of some data structures)
- ***Prototype selection/prototype abstraction*** forms a reduction of the training set which reduces the time and storage requirement by using the reduced prototype set instead of the complete training set.

References

- M. N. Murty and V. S. Devi, 2011, *Pattern Recognition – An Algorithmic Approach*, Springer.
- V. S. Devi and M. S. Murty, 2002, *An Incremental prototype set building technique*, Pattern Recognition, Vol. 35, pp. 505-513.
- X. Xi, E. Keogh, C. Shelton, L. Wei, C. A. Ratanamahatana, *Fast Time Series Classification Using Numerosity Reduction*, Proc. of 23rd Int. Conf. on Machine Learning, Pittsburgh, PA, 2006.

Terminology

- 1-nearest-neighbor: *một lân cận gần nhất*, k-nearest-neighbors: *k-lân cận gần nhất*, distance-weighted k-nearest-neighbors: *k-lân cận gần nhất có trọng số dựa vào khoảng cách*, zero-mean normalization: *chuẩn hóa trung bình zero*, data reduction: *thu giảm dữ liệu*, prototype: *phần tử đại diện*, prototype selection: *lựa chọn prototype*, prototype abstraction: *trích yếu prototype*, minimal distance classifier: *bộ phân lớp khoảng cách tối thiểu*, condensed nearest-neighbor algorithm: *giải thuật lân cận gần nhất cô đọng*, pruning techniques: *các kỹ thuật cắt tỉa*, naïve-rank reduction method: *phương pháp thu giảm dựa vào xếp hạng đơn giản*, threshold: *ngưỡng*